PROJECT REPORT

Video Game Hit Predictor

Classifying commercially successful games with a Decision Tree

Binary classification with a Decision Tree | Dataset: vgsales.csv

Name	Dhruvit
Roll No.	B41
Enrollment No.	24004501210349
Division	D
Project ID	P10
Project Title	Video Game Hit Predictor: Classifying commercially successful games with a Decision Tree

1. Problem Statement

The video game industry releases thousands of titles, but only a small fraction become commercial successes. Publishers and platform holders need an early, data-driven way to tell which games are likely to be "hits" so that marketing and distribution budgets go where they matter.

This project frames that need as a **binary classification** task: given a game's platform, genre, publisher, release year and regional sales figures, predict whether it is a **Hit** (Global_Sales ≥ 1.0 million units, label 1) or a **Not Hit** (label 0). A Decision Tree classifier is used because its rules are easy to read and explain.

Success criteria: because only about one game in eight is a hit, accuracy alone is misleading. The model is judged on precision, recall and F1 for the Hit class, together with the confusion matrix and ROC curve.

2. Dataset Description and Data Dictionary

The data comes from **vgsales.csv**, a table of video games with lifetime sales by region. The raw file has **16,598 rows × 11 columns**, covering **31 platforms**, **12 genres**, **578 publishers** and release years **1980–2020**. After cleaning, **16,291 games** remain. Sales are in millions of units. One column, *Hit*, was engineered as the target.

Column	Type	Description	Role
Rank	int	Rank of the game by global sales	Not used
Name	text	Title of the game (11,493 unique)	Not used
Platform	text	Console / system, 31 values (e.g. Wii, DS, PS2)	Feature
Year	float	Release year, 1980–2020 (271 missing)	Feature
Genre	text	Game genre, 12 values (Action is most common: 3,316)	Feature
Publisher	text	Publishing company, 578 values (58 missing)	Feature
NA_Sales	float	North America sales (millions of units)	Feature
EU_Sales	float	Europe sales (millions of units)	Feature
JP_Sales	float	Japan sales (millions of units)	Feature
Other_Sales	float	Rest-of-world sales (millions of units)	Feature
Global_Sales	float	Worldwide sales (millions of units)	Used to build target
Hit	int (0/1)	Engineered: 1 if Global_Sales ≥ 1.0, else 0	Target

Table 1. Data dictionary (Hit is the engineered target).

3. Data Cleaning and Exploratory Data Analysis

Cleaning and preparation steps

- **Duplicates and missing values:** duplicates were dropped, then rows with missing values were removed (Year: 271, Publisher: 58). This took the data from 16,598 to **16,291** rows, a loss of only 1.8%, so no imputation was needed.
- **Target creation:** Hit = 1 when Global_Sales ≥ 1.0 million. This gives **2,058 hits (12.6%)** and **14,233 non-hits (87.4%)**.
- **Encoding:** Platform, Genre and Publisher were one-hot encoded, giving **624 input features**. Rank and Name were left out as identifiers.
- **Split:** 80% train (13,032 rows) / 20% test (3,259 rows), stratified on the target with random_state = 42, so both sets keep the same 12.6% hit rate.

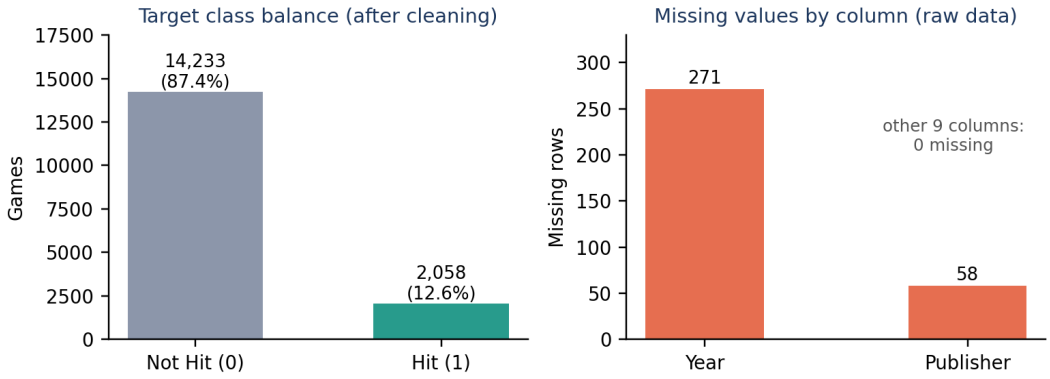


Figure 1. Left: the Hit target is imbalanced. Right: missing values were limited to Year and Publisher.

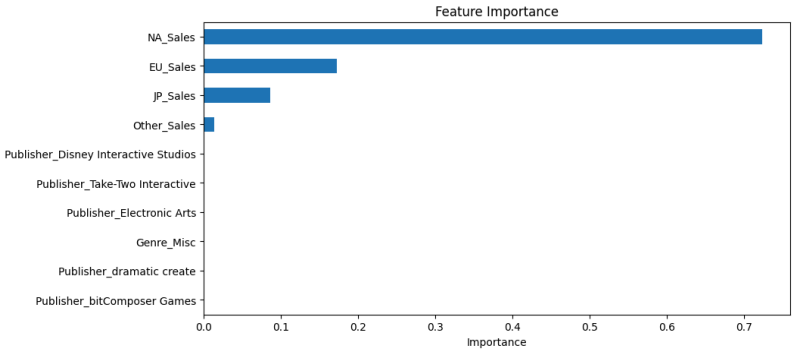


Figure 2. Feature importance of the depth-5 tree (top 10 features).

Key insights

- **Strong class imbalance.** Only 12.6% of games are hits. A model that always said "Not Hit" would score 87% accuracy, so precision, recall and F1 matter more than accuracy here, and the stratified split is essential.
- **Missing data is minor.** Only Year (1.6%) and Publisher (0.4%) had gaps, so dropping those rows was a safe choice.
- **Sales are very skewed.** North America is the biggest market (mean 0.26M per game vs 0.15M in Europe and 0.08M in Japan), and a few blockbusters such as Wii Sports (82.7M) sit far above the typical game, which sells under 1M.

- **Sales columns dominate the model.** In the tree, NA_Sales (≈72%), EU_Sales (≈17%), JP_Sales (≈9%) and Other_Sales (≈1%) account for about 99% of total importance. Platform, genre and publisher contribute almost nothing.

4. Model and Settings Tried

A scikit-learn **DecisionTreeClassifier** was trained on the 13,032 training rows and scored on the 3,259 held-out rows (412 hits, 2,847 non-hits). The default tree was tried first, then the two settings that control complexity were varied: **max_depth** (how deep the tree may grow) and **criterion** (the split quality measure: Gini or entropy). random_state was fixed at 42 for every run.

#	Setting	max_depth	criterion
1	Default model	None (unlimited)	gini
2	Shallow tree	3	gini
3	Medium tree (selected)	5	gini
4	Deeper tree	8	gini
5	Deep tree	10	entropy
6	Unlimited, entropy	None (unlimited)	entropy

Table 2. Six Decision Tree configurations tested (selected setting highlighted).

The notebook picked the setting with the highest **precision**, which was max_depth = 5 with Gini (0.974). It was retrained as the final model.

5. Results

Setting	Accuracy	Precision	Recall	F1
Default (unlimited, gini)	0.991	0.973	0.954	0.963
Depth 3, gini	0.977	0.940	0.876	0.907
Depth 5, gini (final)	0.985	0.974	0.908	0.940
Depth 8, gini	0.990	0.966	0.951	0.958
Depth 10, entropy	0.989	0.965	0.949	0.957
Unlimited, entropy	0.990	0.966	0.951	0.958

Table 3. Test-set metrics for each setting (Hit class for precision, recall and F1).

Reading the table. Very shallow trees (depth 3) under-fit and miss many hits (recall 0.876). From depth 8 onward the scores level off at about 0.99 accuracy and 0.95 recall, so extra depth adds nothing. The final depth-5 tree has the best precision (0.974) but gives up some recall, while the default tree has the best accuracy, recall and F1 overall.

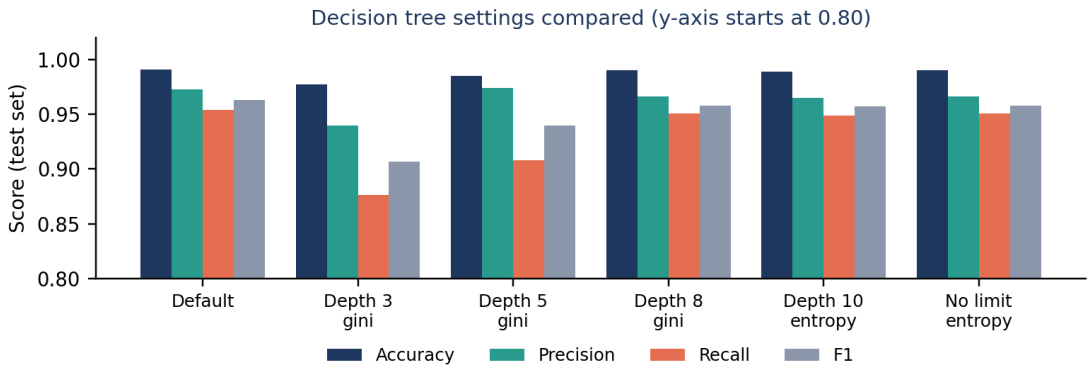


Figure 3. Metric comparison across settings.

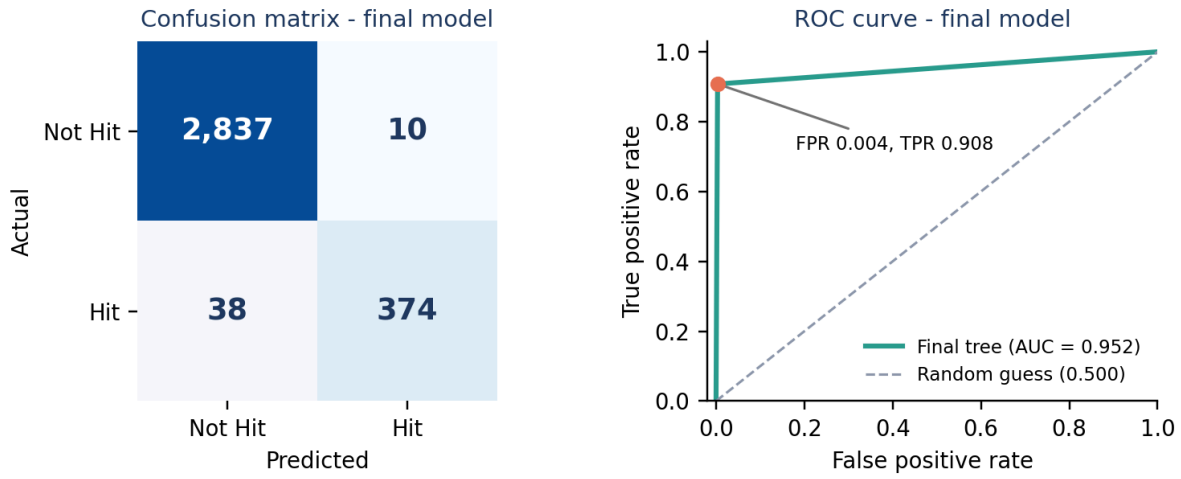


Figure 4. Confusion matrix (left) and ROC curve (right) for the final depth-5 model on the 3,259-row test set.

Confusion matrix (final model). Of 2,847 non-hits, 2,837 were correct and only **10** were wrongly flagged as hits. Of 412 real hits, **374** were found and **38** were missed. False alarms are rare (precision 0.974), while about 9% of hits slip through (recall 0.908).

ROC curve. The curve is drawn from the final model's hard predictions (true-positive rate 0.908, false-positive rate 0.004), giving **AUC ≈ 0.952**. The notebook did not save predicted probabilities, so this is the single-threshold ROC, not a full probability sweep.

6. Conclusion and What I Learned

What the project achieved. A depth-5 decision tree classifies games as Hit or Not Hit with **98.5% accuracy, 0.974 precision, 0.908 recall, F1 0.940 and AUC ≈ 0.95** on unseen data. Tuning showed that depth beyond about 8 gives no further gain, while depth 3 is clearly too simple.

Important limitation: target leakage. Hit is defined from Global_Sales, and Global_Sales is essentially the sum of the four regional sales columns, which were used as inputs. The tree is therefore mostly learning a sales threshold, which is why NA_Sales alone carries 72% of the importance. The scores above are real for this setup but overstate how well the model could *predict* a hit before launch.

Other limitations. The best setting was chosen by precision on the same test set used for reporting, there was no cross-validation, and only one model family was tried.

What I learned. (1) With imbalanced classes, accuracy hides problems, so precision, recall, F1, the confusion matrix and ROC are needed. (2) Tree depth controls the trade-off between under-fitting and over-fitting. (3) One-hot encoding can expand a few columns into hundreds of features (624 here). (4) Check how the target is built before trusting a very high score, since inputs that help define the label make a model look better than it is.

Next steps. Remove all sales columns and predict hits from platform, genre, publisher and year only; use cross-validation and a validation set for tuning; add predict_proba to plot a full ROC curve; and compare with Random Forest or Gradient Boosting.